

CLAUDE CODE & CODEX CLI

Type less, code more

The complete guide to the two leading command-line coding agents: every command, option and shortcut, plus a detailed comparison to help you pick the right tool for the job.

WRITTEN AND VERIFIED • JULY 2026

ISMAEL LADJOHOUNLOU • [linkedin.com/in/ismaelladjohounlou](https://www.linkedin.com/in/ismaelladjohounlou)

Introduction

Claude Code (Anthropic) and Codex CLI (OpenAI) are today's two leading command-line coding agents. Both let you delegate full development tasks (reading code, editing it, running shell commands, searching the web) straight from the terminal, with a high degree of autonomy.

This guide has two goals:

1. Serve as a **complete reference** for Claude Code: every slash command, CLI option, keyboard shortcut, and configuration mechanism (CLAUDE.md, Skills, Hooks, MCP, Subagents, Plan Mode, Agent SDK).
2. Do the same for **Codex CLI**, then offer a **direct comparison** between the two tools to help you choose the one that fits your use case best.

Every command and option listed here is reproduced exactly as it must be typed, including the hyphens (-) that separate words in command and flag names. Do not alter or translate them: `/code-review` and `--permission-mode` must be typed letter for letter.

Part I: Claude Code

ANTHROPIC

What is Claude Code?

Claude Code is Anthropic's command-line coding agent. It runs in the terminal (and also ships as a VS Code extension, a JetBrains plugin, a desktop app, and a web app at claude.ai/code). It reads and edits files, runs shell commands, searches the web, and can orchestrate subagents for complex tasks, all driven by natural-language instructions.

The entry point is the `claude` command in a terminal, launched from a project's root.

Quick start

- `claude` : launches an interactive session in the current folder
- `claude -p "fix the bug in auth.py"` : print mode, a single response then exit
- `claude --continue` : resumes the last conversation in this folder

Slash commands

Slash commands are typed directly into the interactive session. They cover session management, configuration, built-in tools, and more.

SESSION MANAGEMENT

COMMAND	SYNTAX	ROLE
<code>/help</code>	<code>/help</code>	Displays the help menu and list of commands
<code>/clear</code>	<code>/clear [name]</code>	Starts a new conversation with an empty context
<code>/exit</code>	<code>/exit</code> (or Ctrl+D)	Exits the session
<code>/rename</code>	<code>/rename [name]</code>	Renames the current session
<code>/resume</code>	<code>/resume [session]</code>	Resumes a previous conversation by ID or name
<code>/rewind</code>	<code>/rewind</code>	Rewinds to an earlier checkpoint in the conversation
<code>/branch</code>	<code>/branch [name]</code>	Creates a conversation branch to explore another direction
<code>/background</code>	<code>/background [prompt]</code>	Detaches the session as a background agent, freeing the terminal

<code>/remote-control</code>	<code>/remote-control</code>	Makes the session controllable remotely from claude.ai
------------------------------	------------------------------	--

CONTEXT AND NAVIGATION

COMMAND	SYNTAX	ROLE
<code>/cd</code>	<code>/cd <path></code>	Moves the session to a new working directory
<code>/add-dir</code>	<code>/add-dir <path></code>	Adds extra working directories
<code>/context</code>	<code>/context [all]</code>	Visualizes context usage as a colored grid
<code>/compact</code>	<code>/compact [instructions]</code>	Frees up context by summarizing the conversation
<code>/memory</code>	<code>/memory</code>	Views and edits CLAUDE.md files and auto memory
<code>/recap</code>	<code>/recap</code>	Generates a one-line summary of the current session
<code>/focus</code>	<code>/focus</code>	Toggles a focus view showing only key information

MODEL AND CONFIGURATION

COMMAND	SYNTAX	ROLE
<code>/model</code>	<code>/model [model]</code>	Switches model and saves it as the default
<code>/effort</code>	<code>/effort [level auto]</code>	Sets the effort level (low, medium, high, xhigh, max)
<code>/advisor</code>	<code>/advisor [model off]</code>	Enables a second advisor model
<code>/config</code>	<code>/config [key=value ...]</code>	Opens settings or sets a preference directly

CODE REVIEW AND QUALITY

COMMAND	SYNTAX	ROLE
<code>/code-review</code>	<code>/code-review [low medium high xhigh max ultra] [--fix] [--comment] [target]</code>	Reviews the diff for bugs and cleanups
<code>/review</code>	<code>/review [PR]</code>	Fast, read-only review of a GitHub PR
<code>/security-review</code>	<code>/security-review</code>	Analyzes changes for security vulnerabilities
<code>/simplify</code>	<code>/simplify [target]</code>	Looks for simplification opportunities and applies them
<code>/diff</code>	<code>/diff</code>	Opens an interactive diff viewer

TESTING AND VERIFICATION

COMMAND	SYNTAX	ROLE
<code>/run</code>	<code>/run</code>	Launches the app to validate changes
<code>/verify</code>	<code>/verify</code>	Builds and runs to confirm the code works
<code>/run-skill-generator</code>	<code>/run-skill-generator</code>	Teaches <code>/run</code> and <code>/verify</code> how to launch your app
<code>/batch</code>	<code>/batch</code> <code><instruction></code>	Orchestrates large-scale changes across the codebase in parallel

AUTOMATION AND SCHEDULING

COMMAND	SYNTAX	ROLE
<code>/loop</code>	<code>/loop [interval] [prompt]</code>	Repeats a prompt while the session stays open
<code>/schedule</code>	<code>/schedule [description]</code>	Creates, edits, lists, or runs scheduled tasks
<code>/goal</code>	<code>/goal [condition clear]</code>	Sets a condition to work toward across turns

TOOLS AND INTEGRATIONS

COMMAND	SYNTAX	ROLE
<code>/mcp</code>	<code>/mcp [reconnect</code> <code><server> enable disable]</code>	Manages MCP server connections
<code>/chrome</code>	<code>/chrome</code>	Configures the Claude in Chrome extension
<code>/ide</code>	<code>/ide</code>	Manages IDE integrations and shows their status
<code>/plugin</code>	<code>/plugin [subcommand]</code>	Manages Claude Code plugins
<code>/skills</code>	<code>/skills</code>	Lists available skills and manages their visibility
<code>/reload-plugins</code>	<code>/reload-plugins [--force]</code>	Reloads active plugins
<code>/reload-skills</code>	<code>/reload-skills</code>	Rescans skill directories

PERMISSIONS AND ACCESS

COMMAND	SYNTAX	ROLE
---------	--------	------

<code>/permissions</code>	<code>/permissions</code>	Manages tool permission rules (allow/deny/ask)
<code>/hooks</code>	<code>/hooks</code>	Shows hook configuration

DEBUGGING AND DIAGNOSTICS

COMMAND	SYNTAX	ROLE
<code>/debug</code>	<code>/debug [description]</code>	Enables debug logging
<code>/doctor</code>	<code>/doctor</code>	Diagnoses and fixes installation issues
<code>/heapdump</code>	<code>/heapdump</code>	Writes a heap snapshot for diagnostics

ACCOUNT AND INFORMATION

COMMAND	SYNTAX	ROLE
<code>/login</code>	<code>/login</code>	Signs in to your Anthropic account
<code>/logout</code>	<code>/logout</code>	Signs out
<code>/usage</code>	<code>/usage</code>	Tracks API usage and cost
<code>/cost</code>	<code>/cost</code>	Alias for <code>/usage</code>
<code>/status</code>	<code>/status</code>	Opens settings on the Status tab
<code>/feedback</code>	<code>/feedback [report]</code>	Sends feedback or reports a bug

MISCELLANEOUS

COMMAND	SYNTAX	ROLE
<code>/plan</code>	<code>/plan [description]</code>	Enters plan mode directly
<code>/agents</code>	<code>/agents</code>	Manages or creates subagent configurations
<code>/copy</code>	<code>/copy [N]</code>	Copies the last response to the clipboard
<code>/export</code>	<code>/export [file]</code>	Exports the conversation as plain text
<code>/keybindings</code>	<code>/keybindings</code>	Opens the keybindings file for customization
<code>/statusline</code>	<code>/statusline</code>	Configures the status line
<code>/fast</code>	<code>/fast [on off]</code>	Toggles fast mode
<code>/sandbox</code>	<code>/sandbox</code>	Toggles sandbox mode for command execution
<code>/install-github-app</code>	<code>/install-github-app</code>	Installs Claude's GitHub app on a repo
<code>/install-slack-app</code>	<code>/install-slack-app</code>	Installs Claude's Slack app

claude command options

These options are used on the command line, before an interactive session even starts.

SESSION AND MODEL

OPTION	ALIAS	ROLE
<code>--print</code>	<code>-p</code>	Answers once without interactive mode, then exits
<code>--continue</code>	<code>-c</code>	Loads the last conversation in the current folder
<code>--resume</code>	<code>-r</code>	Resumes a specific session by ID or name
<code>--background</code>	<code>--bg</code>	Starts as a background agent
<code>--model</code>		Picks the model by alias or full name
<code>--effort</code>		Effort level (low, medium, high, xhigh, max)
<code>--fallback-model</code>		Chain of fallback models

PERMISSIONS AND SECURITY

OPTION	ROLE
<code>--permission-mode</code>	Permission mode at startup (default, acceptEdits, plan, auto, dontAsk, bypassPermissions)
<code>--dangerously-skip-permissions</code>	Skips all permission prompts
<code>--allowed-tools</code>	Pre-approves specific tools, e.g. <code>"Bash(git *)"</code>
<code>--disallowed-tools</code>	Blocks specific tools, e.g. <code>"Bash(rm *)"</code>
<code>--tools</code>	Restricts the list of available built-in tools

SYSTEM PROMPT

OPTION	ROLE
<code>--system-prompt</code>	Fully replaces the system prompt
<code>--system-prompt-file</code>	Loads the system prompt from a file
<code>--append-system-prompt</code>	Appends text to the end of the default system prompt
<code>--append-system-prompt-file</code>	Appends the content of a file to the system prompt

INPUT/OUTPUT

OPTION	ROLE
<code>--output-format</code>	Output format: text, json, stream-json
<code>--input-format</code>	Input format: text, stream-json
<code>--verbose</code>	Detailed, turn-by-turn logging
<code>--json-schema</code>	JSON output validated against a supplied schema

FILES AND DIRECTORIES

OPTION	ALIAS	ROLE
<code>--add-dir</code>		Adds extra working directories
<code>--worktree</code>	<code>-w</code>	Starts in an isolated Git worktree
<code>--tmux</code>		Creates a tmux session for the worktree

MCP AND PLUGINS

OPTION	ROLE
<code>--mcp-config</code>	Loads MCP servers from a file or inline JSON string
<code>--strict-mcp-config</code>	Only uses MCP servers supplied via <code>--mcp-config</code>
<code>--plugin-dir</code>	Loads a plugin from a folder or a <code>.zip</code> archive

SETTINGS AND DIAGNOSTICS

OPTION	ROLE
<code>--settings</code>	Path to a JSON settings file, or inline JSON
<code>--setting-sources</code>	Comma-separated sources to load (user, project, local, managed)
<code>--safe-mode</code>	Starts with all customizations disabled
<code>--bare</code>	Minimal mode: no auto-detected hooks, skills, plugins, or MCP
<code>--max-turns</code>	Limits the number of agentic turns
<code>--max-budget-usd</code>	Dollar cap on API call spend
<code>--debug</code>	Debug mode, with optional category filtering
<code>--version (-v)</code>	Prints the version number

Keyboard shortcuts

SHORTCUT	ACTION
<code>Shift+Tab</code>	Cycles permission mode (default → acceptEdits → plan → auto → bypassPermissions)
<code>Ctrl+D</code>	Exits the session (same as <code>/exit</code>)
<code>Tab</code>	Command completion
<code>↑</code>	Recalls the previous command from history
<code>Ctrl+C</code>	Interrupts the current operation
<code>Ctrl+L</code>	Clears the screen
<code>Ctrl+R</code>	Opens recent sessions / searches history
<code>Esc Esc</code>	Cancels the current input
<code>Ctrl+U</code>	Clears the current line
<code>Ctrl+W</code>	Deletes the previous word
<code>Ctrl+G</code>	Opens the proposed plan in the default editor for editing

In VS Code: `Ctrl+Shift+A` toggles the Claude Code sidebar, `Ctrl+Shift+C` opens it in a new tab (Cmd on macOS).

Settings hierarchy

Settings combine in a strict priority order, from highest to lowest:

1. **Managed** (enforced by the organization, not overridable): `/etc/claude-code/managed-settings.json` (Linux), Windows registry, or macOS plist
2. **Command-line arguments**
3. **Local**: `.claude/settings.local.json` (gitignored, specific to you on this repo)
4. **Project**: `.claude/settings.json` (shared with the team, committed)
5. **User**: `~/.claude/settings.json` (your preferences, all projects)

Among the most useful keys: `model` , `permissions.allow` / `permissions.deny` , `permissions.defaultMode` , `env` , `hooks` , `autoMemoryEnabled` , `disableBundledSkills` .

CLAUDE.md: persistent memory

CLAUDE.md is a markdown file that gives Claude persistent instructions about a project, your workflow, or an entire organization. Claude reads it at the start of every session.

SCOPE	LOCATION	LOADING
Organization	platform-dependent (e.g. <code>/etc/claude-code/CLAUDE.md</code>)	Always, cannot be excluded
User	<code>~/.claude/CLAUDE.md</code>	Always
Project	<code>./CLAUDE.md</code> or <code>./.claude/CLAUDE.md</code>	Always
Local	<code>./CLAUDE.local.md</code> (gitignored)	Always
Subdirectories	<code>./path/CLAUDE.md</code>	On demand, when Claude reads files in that folder

Files are imported with the `@path/to/file.md` syntax (maximum depth: 4 levels). Path-scoped rules can also be defined in `.claude/rules/*.md` , with a `paths:` frontmatter to limit them to specific files.

Alongside CLAUDE.md there's **auto memory**: Claude notes down, on its own and without being asked, corrections and preferences it observes (stored at `~/.claude/projects/<project>/memory/MEMORY.md`). It can be disabled with `"autoMemoryEnabled": false` .

Skills

A **Skill** is a `SKILL.md` file that extends Claude's capabilities. Unlike CLAUDE.md, its content is only loaded into context when invoked, via `/skill-name` or automatically when Claude judges it relevant.

LOCATION	SCOPE
<code>~/.claude/skills/<name>/SKILL.md</code>	Personal, all your projects
<code>.claude/skills/<name>/SKILL.md</code>	This project only
A plugin's <code>skills/</code> subdirectory	Wherever the plugin is enabled

A Skill's frontmatter accepts `name` , `description` , `argument-hint` , `allowed-tools` , `disallowed-tools` , `model` , `effort` , `context: fork` (runs in an isolated subagent), and `disable-model-invocation: true` to allow manual invocation only.

Hooks

Hooks are user-defined shell commands that run at specific points in Claude Code's lifecycle: deterministic control that applies no matter what the model decides.

EVENT	TRIGGERED	USEFUL FOR
SessionStart	At the start of a session	Initializing the environment, starting servers
Setup	At <code>/init</code> or on startup	Installing dependencies
PreToolUse	Before any tool runs	Validating, blocking an action
PostToolUse	After a tool succeeds	Auto-formatting, notifying
PermissionRequest	When approval is required	Auto-approving certain actions
PostCompaction	After the context is summarized	Re-injecting context

Configured in `settings.json`, with an object like `"if": "Name(pattern)"` and `"run": "command"`.

MCP: Model Context Protocol

MCP is an open standard for connecting Claude Code to external tools, databases, and APIs. An MCP server bridges Claude to a given system.

Added via `/mcp`, or on the CLI with `--mcp-config ./mcp.json`. Configuration file `.mcp.json`:

```
{
  "mcpServers": {
    "github": {
      "command": "npx",
      "args": ["@modelcontextprotocol/server-github"],
      "env": { "GITHUB_TOKEN": "ghp_..." }
    }
  }
}
```

Scopes (local, project, user, managed) merge together, with more local scopes taking precedence over more global ones.

Subagents

Subagents are specialized assistants that handle a task in an isolated context, useful when a subtask would generate a lot of noise (search results, logs) with no value for the main conversation. Each subagent

has its own context, system prompt, and tool access.

Defined in `.claude/agents/<name>.md`, with a frontmatter of `name`, `description`, `model`, `tools`, `permission-mode`. Claude delegates automatically when the description matches the task, or can be invoked explicitly ("Use the code-reviewer agent for...").

Built-in agents: **Explore** (codebase research, read-only), **Plan** (planning), **general-purpose** (default).

Plan Mode

Plan mode is a permission mode where Claude explores and proposes changes without executing them. It reads files and code, then writes a structured plan; nothing is edited or run before your approval.

Activation: `claude --permission-mode plan`, or in a session with `/plan` or `Shift+Tab`.

After approval, several execution modes are offered: fully automatic approval, automatic approval of edits only, or manual review of each step.

Claude Agent SDK

The **Claude Agent SDK** gives you, as a library, access to the same engine that powers Claude Code, to build your own agents in Python or TypeScript on your own infrastructure.

LANGUAGE	PACKAGE	INSTALL
Python	<code>claude-agent-sdk</code>	<code>pip install claude-agent-sdk</code> (Python 3.10+)
TypeScript	<code>@anthropic-ai/claude-agent-sdk</code>	<code>npm install @anthropic-ai/claude-agent-sdk</code>

```
import asyncio
from claude_agent_sdk import query, ClaudeAgentOptions

async def main():
    async for message in query(
        prompt="Find and fix the bug in auth.py",
        options=ClaudeAgentOptions(allowed_tools=["Read", "Edit", "Bash"]),
    ):
        if hasattr(message, "result"):
            print(message.result)

asyncio.run(main())
```

The SDK includes the Read, Write, Edit, Bash, Glob, Grep, WebSearch, WebFetch, and AskUserQuestion tools by default, and automatically loads `.claude/skills/`, `.claude/agents/`, and `CLAUDE.md` from the working directory.

Part II: Codex CLI

OPENAI

What is Codex CLI?

Codex CLI is OpenAI's command-line coding agent. Like Claude Code, it runs in the terminal, reads and edits code, runs shell commands, and can operate autonomously or under supervision. Run with no subcommand, `codex` opens an interactive terminal UI (TUI).

Quick start

`codex` : launches the interactive terminal UI

`codex --sandbox workspace-write --ask-for-approval on-request` : low-friction local work

Built-in slash commands

COMMAND	ROLE
<code>/permissions</code>	Adjusts what Codex can do without asking
<code>/approve</code>	Re-authorizes a recently denied automatic action
<code>/ide</code>	IDE integration
<code>/keymap</code>	Customizes keyboard shortcuts
<code>/vim</code>	Enables Vim-style editing shortcuts
<code>/plugins</code>	Manages Codex plugins
<code>/hooks</code>	Manages Codex hooks
<code>/clear</code>	Clears the current conversation
<code>/rename</code>	Renames the session
<code>/archive</code>	Archives a saved session
<code>/delete</code>	Deletes a session
<code>/compact</code>	Summarizes the conversation to free up context
<code>/copy</code>	Copies Codex's latest completed output
<code>/import</code>	Imports external content into the session
<code>/feedback</code>	Sends feedback to the Codex team

<code>/init</code>	Initializes the project's configuration
<code>/logout</code>	Signs out
<code>/mcp</code>	Manages connected MCP servers
<code>/mention</code>	Mentions a file or resource in the prompt
<code>/model</code>	Switches the active model (e.g. <code>gpt-5.4-mini</code> , <code>gpt-5.5</code>)
<code>/fast</code>	Toggles fast mode
<code>/plan</code>	Switches to planning mode
<code>/app</code>	Application-related actions
<code>/side</code>	Opens a side panel
<code>/btw</code>	Adds a side note without changing topic
<code>/raw</code>	Sends raw, unprocessed input
<code>/resume</code>	Resumes a previous session
<code>/new</code>	Starts a new session
<code>/quit</code>	Exits the CLI (same as <code>/exit</code>)
<code>/review</code>	Asks Codex to review your working tree
<code>/status</code>	Shows session configuration and token usage
<code>/theme</code>	Picks a terminal syntax-highlighting theme
<code>/pets</code> (or <code>/pet</code>)	Picks or hides an animated terminal pet

`/quit` and `/exit` both exit the CLI: use them only after saving or committing your work. `/status` confirms the active model, approval policy, writable directories, and remaining context capacity.

Global flags

FLAG	ALIAS	VALUES / ROLE
<code>--model</code>	<code>-m</code>	Overrides the model set in configuration (e.g. <code>gpt-5.4</code>)
<code>--image</code>	<code>-i</code>	Attaches one or more images to the initial prompt (comma-separated paths)
<code>--local-provider</code>		<code>lmstudio</code> <code>ollama</code> : local provider used with <code>--oss</code>
<code>--no-alt-screen</code>		Disables the TUI's alternate full-screen mode
<code>--oss</code>		Uses a local open-source model

<code>--profile</code>	<code>-p</code>	Layers <code>\$CODEX_HOME/profile-name.config.toml</code> on top of the base config
<code>--remote</code>		Connects to a remote app server over WebSocket or Unix socket (<code>ws://</code> , <code>wss://</code> , <code>unix://</code>)
<code>--remote-auth-token-env</code>		Environment variable holding an auth token for <code>--remote</code>
<code>--sandbox</code>	<code>-s</code>	<code>read-only</code> <code>workspace-write</code> <code>danger-full-access</code> : sandbox policy for model-generated shell commands

Subcommands

COMMAND	STATUS	ROLE
<code>codex app-server</code>	Experimental	Launches the Codex app server (stdio, WebSocket, or Unix socket)
<code>codex apply</code> (alias <code>codex a</code>)	Stable	Applies the latest diff generated by a Codex cloud task
<code>codex archive</code>	Stable	Archives a saved interactive session by ID or name
<code>codex cloud</code> (alias <code>codex cloud-tasks</code>)	Experimental	Browses or runs Codex cloud tasks from the terminal
<code>codex completion</code>	Stable	Generates shell completion scripts (Bash, Zsh, Fish, PowerShell)
<code>codex resume</code>	Stable	Resumes a saved session
<code>codex fork</code>	Stable	Duplicates an existing session into a new one
<code>codex delete</code>	Stable	Deletes a saved session
<code>codex unarchive</code>	Stable	Unarchives a session

To connect the TUI to an already-running app server: `codex app-server --listen ws://IP:PORT` , then connect with `--remote ws://host:port` . Locally, use `--remote unix://` for the default socket.

Interactive shortcuts

SHORTCUT	ACTION
<code>@</code>	Searches for a file in the workspace and adds its path to the prompt
<code>↑ / ↓</code>	Restores draft history
<code>Ctrl+R</code>	Searches prompt history, <code>Enter</code> to use a match, <code>Esc</code> to cancel

<code>Ctrl+O</code> (or <code>/copy</code>)	Copies Codex's latest completed output
<code>!</code> at the start of a line	Runs a local shell command under the current sandbox settings
<code>Tab</code> (while running)	Queues a prompt, slash command, or shell command for the next turn
<code>Enter</code> (while running)	Injects new instructions into the current turn
<code>Esc Esc</code> (empty composer)	Edits the last user message and forks the conversation from that point
<code>Ctrl+C</code> (or <code>/exit</code>)	Closes the session

Configuration

Codex uses two main mechanisms:

- `AGENTS.md` : the CLAUDE.md equivalent, persistent project instructions at the repo root.
- `~/.codex/config.toml` : global configuration (default model, sandbox, providers, profiles).

A **profiles** system lets you layer multiple configurations (`--profile name` loads `$CODEX_HOME/profile-name.config.toml` on top of the base config), useful for quickly switching between "strict" and "permissive" setups, for instance.

Part III: Comparison

Claude Code vs Codex CLI

ASPECT	CLAUDE CODE	CODEX CLI
Publisher	Anthropic	OpenAI
Models	Claude family (Sonnet, Opus, Haiku)	GPT family (e.g. gpt-5.4, gpt-5.5)
Project memory file	CLAUDE.md	AGENTS.md
Global config	~/.claude/settings.json (JSON)	~/.codex/config.toml (TOML)
Extensibility	Skills (SKILL.md), Subagents (.claude/agents/), Plugins	Codex plugins, config profiles
External tool integration	MCP (.mcp.json , /mcp)	MCP (/mcp)
Shell automation	Hooks (SessionStart, PreToolUse, PostToolUse, etc.)	Codex hooks (/hooks)
Planning mode	Plan Mode (--permission-mode plan , Shift+Tab)	/plan
Sandbox	--permission-mode , --allowed-tools / --disallowed-tools	--sandbox (read-only / workspace-write / danger-full-access)
Remote session	Remote Control from claude.ai, --teleport	--remote (WebSocket / Unix socket) to codex app-server
Programmable SDK	Claude Agent SDK (Python / TypeScript)	Codex exposes an app-server as a local API
Local / open-source models	Not native	--oss , --local-provider (LM Studio, Ollama)
Terminal mascot	none	/pets (animated terminal pet)

Respective strengths

CLAUDE CODE

The richest extensibility ecosystem to date: invokable Skills (by you or automatically by the model), Subagents with isolated context and permissions, Hooks at seven lifecycle points, and a full SDK for

building your own agents in production. Auto memory (beyond CLAUDE.md) lets the tool keep learning your preferences without intervention.

CODEX CLI

A lightweight, fast terminal interface, with a TOML configuration profile system that's handy for switching between contexts (e.g. a restrictive "prod" profile vs a permissive "exploration" profile). Native support for local/open-source models (`--OSS` , LM Studio, Ollama) makes it a relevant choice for offline work or private infrastructure.

Which tool for which use case?

Choose Claude Code if: you want to build complex, reusable workflows (Skills, Subagents, Hooks), automate tasks programmatically via the SDK, or benefit from a mature plugin and MCP ecosystem.

Choose Codex CLI if: you prefer a lightweight CLI tightly integrated with the OpenAI ecosystem, need to run models locally for cost or privacy reasons, or prefer TOML-based profile configuration.

In practice, the two tools are close enough in philosophy (autonomous agent, graduated permissions, persistent memory via a markdown file at the project root) to be used side by side depending on the task: nothing stops you from having both `CLAUDE.md` and `AGENTS.md` in the same repo.

Quick cheat sheet

CLAUDE CODE: THE ESSENTIALS

```
claude                                # interactive session
claude -p "question"                  # single response, no interaction
claude --continue                      # resumes the last session
claude --resume session-name          # resumes a specific session
claude --permission-mode plan         # starts in plan mode
claude --model claude-opus-4-8        # model selection
claude -w feature-x                   # isolated Git worktree
/compact                              # frees up context
/model                                # switches model mid-session
Shift+Tab                             # cycles permission mode
Ctrl+R                                # session history
```

CODEX CLI: THE ESSENTIALS

```
codex                                # interactive interface
codex --sandbox workspace-write \
    --ask-for-approval on-request      # low-friction local work
codex -m gpt-5.5                       # model selection
codex --oss --local-provider ollama    # local model via Ollama
codex resume                           # resumes a session
codex apply                            # applies a cloud task's diff
/model                                 # switches model mid-session
/status                                # config + token usage
@                                      # references a file in the
prompt
Ctrl+R                                # prompt history
```